Retrospective: Effects of the WG21 Train Model

Document Number:	P4131R0
Date:	2026-03-21
Audience:	WG21

Reply-to:	Vinnie Falco vinnie.falco@gmail.com C++ Alliance Proposal Team

Table of Contents

Abstract

Revision History

- R0: March 2026 (post-Croydon mailing)

1. Disclosure

2. The Model

3. C++20 - The First Stress Test

- 3.1 Contracts Pulled
- 3.2 Ranges Shipped Knowingly Incomplete
- 3.3 Coroutines Without Library Support
- 3.4 The Defect Load

4. C++23 - The Pandemic Cycle

- 4.1 P2300 Deferred
- 4.2 Contracts Missed Again
- 4.3 What C++23 Delivered

5. C++26 - Maximum Stress

- 5.1 Contracts (P2900)
- 5.2 `std::execution::task` (P3552)
- 5.3 `std::execution` (P2300)

6. The Implementers Speak

- 6.1 The Assessment
- 6.2 The Suggested Actions

6.3 Selective Non-Compliance
7. Social Dynamics
7.1 The Persistence of Deadline Pressure
7.2 The Cost of Pulling
7.3 The Vasa Warning
7.4 Committee Scale
7.5 The "Ship It and Fix It" Culture
8. What the Model Delivers
9. What the Model Does Not Deliver
10. Summary
11. Limitations
Acknowledgements
References

Abstract

The train model promised that features would be pulled when not ready. The record shows they are pushed when almost ready.

This paper surveys the published record of the three-year release cadence from C++14 through C++26. It documents the model's claims against outcomes for each release, examines three contested C++26 features (contracts, `std::execution::task`, `std::execution`), and presents the implementers' own assessment that features accumulate faster than they can be implemented. Every exhibit is drawn from published papers, meeting minutes, blog posts, or public testimony. The conclusions are the reader's.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The author has papers in the WG21 mailing and participates in the committee process. The author developed and maintains Corosio[1] and Capy[2] and is a co-author of P2469R0[3]. The author's preferred asynchronous model competes with P2300R10[4] (`std::execution`). The analysis in this paper is structural, not personal. Every decision examined was made by experienced practitioners under real constraints. The intent is to document the published record, not to assign blame.

This paper is a companion to P4129R1[5], which documents the dynamics of WG21 voting, and P4130R0[6], which documents the convenership's structural influence over WG21. Those papers examine the mechanism and the officer. This paper examines the schedule.

The author provides information, asks nothing, and serves at the pleasure of the chair.

---

2. The Model

In 2011, at the Bloomington meeting, Herb Sutter proposed a change to the committee's release cadence. The minutes record the rationale[7]:

> *Knowing that another ship train is coming is a nice pressure release valve. The biggest problem with 0x was feature creep - the mentality that if a feature does not get in now it never will. The now or never pressure makes adding stuff more urgent and creates tension in the committee.*

The committee adopted the proposal. C++ would ship every three years. The model was documented in P1000R2[8] and revised through P1000R6[9]. Its core claim is a single sentence[8]:

> *Model (2) simply means "ship what's ready."*

The safety valve is explicit[9]:

> *If we see a feature is not ready yet, we pull it back out to let it bake more, as we did for concepts in C++11 and contracts in C++20. It can finish baking and ship in C++next.*

The model's author assessed its early results on his blog in 2016[10]:

> *Never before in the history of C++ has the standard maintained such high specification quality, never before has it released on a regular predictable schedule... and as we removed that quality and schedule uncertainty, it's no accident that never before have all the major compilers tracked the standard so closely, which benefits the whole worldwide C++ community.*

And in 2019[11]:

> *By objective metrics, notably national body comment volume and defect reports, C++14 and C++17 have been our most stable releases ever, each about 3-4 times better on those metrics than C++98 or C++11.*

The same blog post acknowledged that the major/minor alternation did not materialize[10]:

> *Also, we initially thought a train model would let us alternate "minor" and "major" releases. The idea was that C++14 could be a "minor" release, and C++17 a "major" release. What actually happened was that we learned the pipeline+train model leads to regular "medium" releases as the pipeline moves at a steady pace and we regularly release what's ready.*

The model makes five testable claims:

Claim	Source
Ship what is ready	P1000R2-R6[8]
Pull features that are not ready	P1000R6[9]
Eliminate "now or never" pressure	N3316[7]
Higher specification quality	Sutter blog[10]
Closer compiler tracking	Sutter blog[10]

The following sections examine the published record for each release under this model.

---

3. C++20 - The First Stress Test

C++20 was the first release under the train model to carry major language features: modules, coroutines, concepts, ranges, and contracts. Four of the five shipped. The fifth tested the safety valve.

3.1 Contracts Pulled

At Cologne in July 2019, the committee removed contracts from C++20. Josuttis, Voutilainen, Orr, Vandevoorde, Spicer, and Di Bella wrote in P1823R0[12]:

> *This is a proposal nobody likes and we are not happy to make it. But we have to suggest to remove Contracts from C++20, because the feature is not ready for standardization yet and to continue to force to have it in C++20 is taking the whole C++20 at risk.*

The plenary vote was 68 in favour, 0 opposed, 4 abstaining[12]. The safety valve worked.

The feature then consumed committee bandwidth across four consecutive cycles: removed from C++20 in 2019, redesigned through C++23, contested again in C++26. The pull was not free.

3.2 Ranges Shipped Knowingly Incomplete

Revzin, Hoekstra, and Song wrote in P2214R0[13]:

> *When Ranges was merged into C++20, it was knowingly incomplete. While it was based on the implementation experience in range-v3, only a small part of that library was adopted into C++20.*

"Ship what is ready" became "ship what is ready enough." The remainder was planned for C++23.

3.3 Coroutines Without Library Support

C++20 shipped coroutines as a language feature without a standard task type, without library-level coroutine utilities, and without integration with the rest of the standard library. Users who wanted to use coroutines needed a third-party library or had to write their own promise types. The C++20 standard provides the coroutine machinery - `coroutine_handle`, `coroutine_traits`, `suspend_always`, `suspend_never` - but no concrete coroutine type that a user can pick up and use.

A standard `std::execution::task` type would not be proposed until P3552[14], six years later, targeting C++26.

3.4 The Defect Load

By 2021, the consequences were visible. Nina Ranns wrote in the admin telecon minutes (N4890)[15]:

> *This time we have quite a few "defects" in C++20 that are too large to be handled via the issues list, so they are being fixed by proposals... At this time, there are no nonexperimental implementations of these features shipped, so this is our last chance to get them fixed before the defects ship.*

The model's author had written in P1000R2[8]:

> *The reality is that we aren't going to find many feature interaction problems until users are using them in production, which generally means until after the standard ships, because implementers will generally wait until the standard ships to implement most things.*

Two years after C++20 shipped, some of its features had no nonexperimental implementations. The defects were proposal-sized.

---

4. C++23 - The Pandemic Cycle

C++23 was developed almost entirely during the COVID-19 pandemic. The committee met virtually from 2020 through early 2022. The Direction Group assessed the impact in P2000R4[16]:

> *The attendance of virtual meetings is less consistent (several key voices are often missing)... the proposals that do make it through the new processes are mostly modest.*

4.1 P2300 Deferred

The sender/receiver framework (P2300[4]) was proposed for C++23. The January 2022 electronic poll to send P2300R4 to LWG received SF:23 / WF:14 / N:0 / WA:6 / SA:11 - no consensus[17]. The published outcome noted:

> *Sustained strong opposition against including such a large proposal into C++23 at such a late stage.*

One voter comment[17]:

> *This is an enormous proposal that is less than 6 months old.*

P2300 was deferred to C++26.

4.2 Contracts Missed Again

Contracts missed C++23 entirely. The Direction Group wrote in P0939R4[18]:

> *We would prefer a delay until C++26 over another heated debate and potential failure for C++23.*

Pulled from C++20 in 2019. Absent from C++23 in 2023. The feature that tested the safety valve was now in its second consecutive cycle without a home.

4.3 What C++23 Delivered

C++23 shipped on time. It delivered `std::expected`, `std::print`, `std::mdspan`, `std::generator`, deducing `this`, `std::flat_map`, and ranges completions planned since C++20. These were valuable additions. The release was not contested.

The train model worked for C++23 in the way it was designed to work: the large, contested features were deferred, and the smaller, ready features shipped. The Direction Group's own word for the result was "modest."

---

5. C++26 - Maximum Stress

C++26 carries three features that each generated substantial published opposition: contracts (P2900), `std::execution` (P2300), and `std::execution::task` (P3552). All three are in the working paper. All three produced paper trails documenting concern about readiness.

5.1 Contracts (P2900)

Contracts were pulled from C++20 in 2019 (Section 3.1). They missed C++23 entirely (Section 4.2). They returned as P2900[19] targeting C++26.

The published opposition is extensive:

- P3573R0[20] (Stroustrup and eight co-authors)
- P3829R0[21] (Chisnall, Spicer, Dos Reis, Voutilainen, Garcia Sanchez): "Contracts do not belong in the language"
- P3835R0[22], P3851R0[23], P3849R1[24] (Swedish NB)
- P3853R0[25], P3878R0[26] (Voutilainen, Wakely, Spicer)

National body comments from the C++26 CD ballot (N5028)[27] include ES-048, ES-049, ES-050, US 25-052, US 26-051, FR-004, FR-005, FI-071, and RO 2-056.

The defense is documented in P3846R0[28] (Doumler, Berne, and fifteen co-authors), which addresses seventeen numbered concerns. At Hagenberg in February 2025, EWG polled "remove P2900 from CWG's consideration for C++26" and reached consensus against removal[29].

Ville Voutilainen wrote in P3608R0[30]:

> *And all this with our noses pressed against a deadline. We are rushing things in, without employing prudent engineering practices, most importantly the quality assurance ones, quite plainly meaning especially testing.*

And in P3853R0[25]:

> *We can't afford to ship a contracts facility in a C++ IS without running that experiment. Our users can't afford that. Our ecosystem can't afford that.*

Doumler, Berne, and co-authors wrote in P3912R0[31]:

> *Rushing a new feature with new syntax into C++26 during the NB comment phase runs counter to the established WG21 model for standardisation as described in [SD-4].*

The three-cycle pattern:

Cycle	Contracts Status
C++20	Pulled (P1823R0, plenary 68/0/4)
C++23	Absent
C++26	In working paper, contested

5.2 `std::execution::task` (P3552)

P3552R3[14] proposes `std::execution::task`, a coroutine task type for use with `std::execution`. Jonathan Müller wrote in P3801R0[32]:

> *I championed the plenary objections: First off, the proper procedure was not followed: The paper was forwarded after the feature freeze deadline in a telecon.*

And[32]:

> *If something has issues, and we know that it has issues, we should not have allowed a vote to approve it.*

Dietmar Kühl, the author of P3552, wrote in P3796R1[33]:

> *The argument that we should only standardise perfect design would actually lead to hardly anything getting standardised because pretty much everything is imperfect on some level.*

The model's safety valve, from P1000R6[9]:

> *If we see a feature is not ready yet, we pull it back out to let it bake more.*

Kühl's response[33]:

> *Pretty much everything is imperfect on some level.*

5.3 `std::execution` (P2300)

P2300R10[4] (`std::execution`) was adopted into the C++26 working paper at St. Louis in July 2024[34]. Baker, Niebler, Shoop, and Radu had written in P3109R0[35]:

> *However, before we ship P2300 and freeze its design in the standard, we should ensure that all potentially-breaking design questions are fully resolved, and that the initial release includes a sufficiently well-rounded set of utilities to be generally useful.*

In the C++23 cycle, a voter comment on the P2300R4 poll stated[17]:

> *This is an enormous proposal that is less than 6 months old.*

Deferred from C++23 as "not sufficiently stable." Adopted for C++26 with a companion task type forwarded after the feature freeze deadline.

---

6. The Implementers Speak

At the Kona 2025 meeting, a group of implementers convened a closed session to discuss their challenges. Twenty attended in person. Nine attended remotely. The resulting paper, P3962R0[36], is authored by eighteen implementers from GCC, Clang, MSVC, EDG, libc++, and libstdc++.

6.1 The Assessment

Ranns, Keane, Serebrennikov, Ballman, Sandoe, Caves, DaCamara, Dos Reis, Brito, Meerwald, Xu, Yaghmour, Miller, Childers, Waffl3x, Cardoso Lopes, Tong, and Dionne write in P3962R0[36]:

> *Full conformance to recent standards remains difficult in practice, with some implementations still working toward C++20 conformance with limited capacity to adopt newer standards.*

And[36]:

> *When features accumulate faster than they can be fully implemented, integrated, and adjusted, they stack on top of incomplete or inconsistent foundations, further increasing the risk of non-portable code.*

On the cost of features[36]:

> *Implementation cost is not limited to initial development. Ongoing maintenance, performance implications, ABI stability, testing matrix growth, and interactions with existing features all contribute substantially to the long-term burden.*

On displacement[36]:

> *Adding new features to the standard necessarily displaces other work, including bug fixes, conformance improvements, performance tuning, and various high-value library enhancements.*

On the implementer voice[36]:

> *Implementers' role and expertise is not always fully reflected in committee processes, particularly during early design discussions. Implementation feedback is often introduced late, treated as adversarial, or framed primarily as an obstacle to progress rather than as essential design input.*

6.2 The Suggested Actions

The implementers' recommendations[36]:

> *We would like the committee to consider ways of slowing down the addition of features into the standard to allow implementers to catch up, and to allow the existing features to improve in quality.*

And[36]:

> *The committee should consider longer standardization cycles or alternating feature-focused and consolidation-focused releases.*

And[36]:

> *We should also explicitly prioritize defect resolution, conformance, and portability work alongside new features.*

The model's safety valve operates on individual features: "If we see a feature is not ready yet, we pull it back out"[9]. The implementers are not asking for individual features to be pulled. They are asking for the pace itself to slow.

6.3 Selective Non-Compliance

Daveed Vandevoorde, a senior EDG implementer, wrote in P3590R0[37]:

> *I suspect most of the principal C++ implementations will opt not to be standard compliant for the better part of a decade, while they (a) keep working on outstanding pre-C++26 language features ("modules" being most notable) and (b) prioritize newer C++26 and C++29 features that provide "more bang for the buck."*

A senior implementer predicting that compiler vendors will choose which parts of the standard to implement and which to defer indefinitely. The standard ships. The implementations do not follow.

---

7. Social Dynamics

The train model is a scheduling mechanism. It operates inside a social system. The published record documents five patterns in that system.

7.1 The Persistence of Deadline Pressure

The train model was designed to eliminate "now or never" pressure. Sutter's original rationale at Bloomington in 2011[7]:

> *The biggest problem with 0x was feature creep - the mentality that if a feature does not get in now it never will.*

Fourteen years later, Voutilainen wrote in P3608R0[30]:

> *And all this with our noses pressed against a deadline.*

Müller wrote in P3801R0[32] that `std::execution::task` was forwarded after the feature freeze deadline. The three-year cycle replaced "now or never" with "now or three years from now." Three years is long enough to create real urgency.

7.2 The Cost of Pulling

The model assumes pulling is cheap. The contracts record suggests otherwise.

Contracts were pulled from C++20 in July 2019. The feature spent six years in redesign. It returned as P2900 for C++26 and generated at least nine opposition papers, multiple national body comments, and a plenary-level contest. Stroustrup wrote in P1711R0[38]:

> *Status quo represents a set of tradeoffs among the concerns of people involved.*

Pulling a feature resets the tradeoffs. The people involved change. The concerns change. The new design must build consensus from scratch, against the memory of the old design's failure. The published record shows this process consuming committee bandwidth across four consecutive cycles.

7.3 The Vasa Warning

Bjarne Stroustrup wrote in P1962R0[39]:

> *Remember the Vasa! In fact, I think that the flood of new proposals has increased since I wrote [Vasa]. I think we are trying to do too much too fast. We can do much or do less fast. We cannot do both and maintain quality and coherence.*

The Direction Group invoked the same analogy in P2000R3[40]:

> *In the years leading up to the 1998 standard, the committee members often reminded themselves of the story of the Vasa, the beautiful 17th century battleship that sank on its maiden voyage because of (among other things) insufficient work on its foundation and excessive late additions.*

The same paper[40]:

> *We have no shared aims, no shared taste.*

The 2023 admin telecon minutes (N4942)[41] documented concern about:

> *The impact of the organization of WG21 on design directions. The size of the committee and the hybrid meetings increase the problems related to coherence.*

7.4 Committee Scale

WG21 has grown from approximately 50 members in the 1990s to over 250 active participants. The Direction Group acknowledged in P0939R4[18]:

> *We are a set of interrelated committees... some "design by committee" is unavoidable.*

More participants means more proposals, more competing interests, and more difficulty achieving genuine consensus. The train model's three-year cadence interacts with this scale: each cycle must process a larger volume of proposals through a larger committee, under the same fixed deadline.

7.5 The "Ship It and Fix It" Culture

The model's author wrote in P1000R6[9]:

> *We do intentionally ship features we plan to further improve.*

Kühl wrote in P3796R1[33]:

> *Pretty much everything is imperfect on some level.*

Müller wrote in P3801R0[32]:

> *They and a majority of WG21 believe that it is better to have something now.*

Ranns documented in N4890[15] that C++20 defects were "too large to be handled via the issues list."

The model says pull. The culture says ship. When the two conflict, the published record shows the culture winning.

---

8. What the Model Delivers

The train model provides real benefits. The published record supports four.

Schedule predictability. C++14 shipped in 2014. C++17 shipped in 2017. C++20 shipped in 2020. C++23 shipped in 2023. Four consecutive on-time releases. No previous era of C++ standardization achieved this. The five-year model produced C++98 (1998), then C++03 (2003, a bug-fix release), then a nine-year gap to C++11 (2011). The train runs on time.

Compiler tracking. Sutter wrote in 2016[10]: "never before have all the major compilers tracked the standard so closely." By C++17, major compilers were shipping conforming implementations within months of publication. By C++20, multiple compilers shipped partial conformance the same year. This is a genuine achievement of the predictable cadence.

Reduced pressure for small features. The train model works well for features that are ready. `std::optional`, `std::variant`, `std::filesystem`, `std::expected`, `std::print`, `std::mdspan` - these shipped without controversy, on schedule, across multiple releases. The three-year cadence gave authors a concrete target and a credible "next train" if the feature needed more time.

Early-cycle quality. Sutter wrote in 2019[11]: "C++14 and C++17 have been our most stable releases ever, each about 3-4 times better on those metrics than C++98 or C++11." C++14 and C++17 were the first two releases under the model. They carried smaller feature loads. They were stable.

---

9. What the Model Does Not Deliver

The published record documents five properties the model was expected to provide but did not.

Elimination of rushing pressure. The model was designed to eliminate "now or never"[7]. The C++26 record shows "noses pressed against a deadline"[30], a task type forwarded after the feature freeze[32], and a contracts facility contested through the NB comment phase[31]. The pressure changed form. It did not disappear.

Low-cost pulling. The model assumes that pulling a feature is a minor scheduling adjustment: "It can finish baking and ship in C++next"[9]. Contracts were pulled from C++20 and consumed committee bandwidth across four consecutive cycles. The social cost of pulling - resetting tradeoffs, rebuilding consensus, defending against the memory of failure - is not accounted for in the model.

Coherence. The Direction Group wrote: "We have no shared aims, no shared taste"[40]. Stroustrup warned: "We cannot do both and maintain quality and coherence"[39]. The three-year cadence does not address coherence. It addresses scheduling.

Implementation capacity. Eighteen implementers wrote in P3962R0[36] that "full conformance to recent standards remains difficult in practice" and asked the committee to "consider ways of slowing down the addition of features." The model ships the standard on time. It does not ensure the standard can be implemented on time.

Quality assurance for large features. C++20 produced proposal-sized defects[15]. C++26 carries three contested features with extensive published opposition. The model's quality mechanism is the safety valve: pull what is not ready. The record shows the valve is rarely used for large features because the social cost of pulling exceeds the social cost of shipping.

---

10. Summary

Release	Major Features Shipped	Pulled / Deferred	Contested at Shipping	Defect Load
C++14	Generic lambdas, variable templates			Low
C++17	`std::optional` , `std::variant` , fold exprs	Concepts TS (not merged)		Low
C++20	Modules, coroutines, concepts, ranges	Contracts	Coroutines (NB opposed)	Proposal-sized
C++23	`std::expected` , `std::print` , `std::mdspan`	P2300, contracts		Moderate
C++26	Contracts, `std::execution` , reflection	Trivial relocation (removed)	Contracts, task, P2300	To be determined

The empty cells in the "Contested at Shipping" column for C++14, C++17, and C++23 are findings. The non-empty cells for C++20 and C++26 are also findings.

---

11. Limitations

This paper presents the published record. It does not claim the record tells the whole story.

The train model's benefits are real. Schedule predictability, compiler tracking, and reduced pressure for small features are genuine achievements documented by the model's author and confirmed by the release history. This paper does not argue that the model should be abandoned.

The contested features in C++26 may prove successful. Contracts may ship, stabilize, and become indispensable. `std::execution` may become the foundation of C++ asynchronous programming. `std::execution::task` may prove adequate. The opposition papers document concerns at a point in time. The long-term outcome is not yet known.

The implementers' assessment in P3962R0 reflects the perspective of eighteen individuals. Other implementers may disagree. The paper itself notes that "this is not an extensive list of topics discussed."

Simpler explanations may exist for every pattern documented here. The pandemic disrupted the C++23 cycle. C++20 was the first release to carry multiple major language features under the train model. C++26 may be an outlier rather than a trend. The reader should consider these alternatives.

The conclusions are the reader's.

Acknowledgements

The author thanks Nina Ranns and the seventeen co-authors of P3962R0[36] for documenting the implementers' assessment in a public paper. The author thanks Joaquín M López Muñoz and Peter Dimov for their critique of P4129R1[5], which informed the analytical approach used in this paper. The author thanks Herb Sutter for documenting the train model's rationale and claims in P1000R2[8] through P1000R6[9] and on his blog, providing the baseline against which the record can be examined.

References

- [1] Corosio: https://github.com/cppalliance/corosio
- [2] Capy: https://github.com/cppalliance/capy
- [3] P2469R0: "Asio and Sender/Receiver." https://wg21.link/p2469r0
- [4] P2300R10: "`std::execution`." Dominiak, Evtushenko, Baker, Teodorescu, Howes, Shoop, Garland, Niebler, Lelbach. https://wg21.link/p2300r10
- [5] P4129R1: "The Dynamics of Voting in WG21." Falco. https://wg21.link/p4129r1
- [6] P4130R0: "The Convenership's Structural Influence Over WG21." Falco. https://wg21.link/p4130r0
- [7] N3316: "Minutes, PL22.16 Meeting No. 57, WG21 Meeting No. 52, 15-19 August 2011 Bloomington, Indiana, USA." Kloepper. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2011/n3316.pdf
- [8] P1000R2: "C++ IS schedule." Sutter. https://wg21.link/p1000r2
- [9] P1000R6: "C++ IS schedule." Sutter. https://wg21.link/p1000r6
- [10] Sutter, H. "Trip report: Winter ISO C++ standards meeting." Blog post, March 11, 2016. https://herbsutter.com/2016/03/11/trip-report-winter-iso-c-standards-meeting/
- [11] Sutter, H. "Draft FAQ: Why does the C++ standard ship every three years?" Blog post, July 13, 2019. https://herbsutter.com/2019/07/13/draft-faq-why-does-the-c-standard-ship-every-three-years/
- [12] P1823R0: "Remove Contracts from C++20." Josuttis, Voutilainen, Orr, Vandevoorde, Spicer, Di Bella. https://wg21.link/p1823r0
- [13] P2214R0: "A Plan for C++23 Ranges." Revzin, Hoekstra, Song. https://wg21.link/p2214r0
- [14] P3552R3: "`task` - An Asynchronous Coroutine Task Type." Kühl. https://wg21.link/p3552r3
- [15] N4890: "WG21 2021-05 Admin telecon minutes." Ranns. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/n4890.pdf

- [16] P2000R4: "Direction for ISO C++." Stroustrup, Hinnant, Orr, Vandevoorde, Wong. https://wg21.link/p2000r4
- [17] P2459R0: "2022 January Library Evolution Poll Outcomes." https://wg21.link/p2459r0
- [18] P0939R4: "Direction for ISO C++." Stroustrup, Sutter, Vandevoorde. https://wg21.link/p0939r4
- [19] P2900R10: "Contracts for C++." Berne, Doumler. https://wg21.link/p2900r10
- [20] P3573R0: "Contracts concerns." Stroustrup et al. https://wg21.link/p3573r0
- [21] P3829R0: "Contracts do not belong in the language." Chisnall, Spicer, Dos Reis, Voutilainen, Garcia Sanchez. https://wg21.link/p3829r0
- [22] P3835R0: "Contracts make C++ less safe." Spicer et al. https://wg21.link/p3835r0
- [23] P3851R0: "Position on contracts assertion for C++26." Garcia et al. https://wg21.link/p3851r0
- [24] P3849R1: "Contracts concerns from the Swedish NB." Achitz. https://wg21.link/p3849r1
- [25] P3853R0: "Remove contracts, ship as TS." Voutilainen. https://wg21.link/p3853r0
- [26] P3878R0: "Contracts not fit for hardening." Voutilainen, Wakely, Spicer. https://wg21.link/p3878r0
- [27] N5028: "C++26 CD Ballot Summary." Sutter. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5028.pdf
- [28] P3846R0: "C++26 Contracts, reasserted." Doumler, Berne et al. https://wg21.link/p3846r0
- [29] P2899R1: "Contracts for C++ - Rationale." Doumler et al. Hagenberg 2025 EWG poll. https://wg21.link/p2899r1
- [30] P3608R0: "C++26 is not ready." Voutilainen. https://wg21.link/p3608r0
- [31] P3912R0: "Rushing a new feature." Doumler et al. https://wg21.link/p3912r0
- [32] P3801R0: "Concerns about `std::execution::task`." Müller. https://wg21.link/p3801r0
- [33] P3796R1: "`std::execution::task` issues." Kühl. https://wg21.link/p3796r1
- [34] Sutter, H. "Trip Report: Summer ISO C++ Standards Meeting (St Louis, MO, USA)." Blog post, July 2, 2024. https://herbsutter.com/2024/07/02/trip-report-summer-iso-c-standards-meeting-st-louis-mo-usa/
- [35] P3109R0: "A plan for `std::execution` for C++26." Baker, Niebler, Shoop, Radu. https://wg21.link/p3109r0
- [36] P3962R0: "Implementation reality of WG21 standardization." Ranns, Keane, Serebrennikov, Ballman, Sandoe, Caves, DaCamara, Dos Reis, Brito, Meerwald, Xu, Yaghmour, Miller, Childers, Waffl3x, Cardoso Lopes, Tong, Dionne. https://wg21.link/p3962r0
- [37] P3590R0: "Constexpr Coroutines Burdens." Vandevoorde. https://wg21.link/p3590r0
- [38] P1711R0: "What to do about contracts?" Stroustrup. https://wg21.link/p1711r0
- [39] P1962R0: "How can you be so certain?" Stroustrup. https://wg21.link/p1962r0
- [40] P2000R3: "Direction for ISO C++." Stroustrup, Hinnant, Orr, Vandevoorde, Wong. https://wg21.link/p2000r3
- [41] N4942: "WG21 2023-06 Admin telecon minutes." Ranns. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4942.pdf